

## [H-1] Storing the password onchain makes it visible to anyone

**Description:** All data stored onchain is visible to anyone and can be accessed by anyone. The `PasswordStore::s_password` variable is intended to be private and only accessed by the `PasswordStore::getPassword` function, which is intended to be called by the owner of the contract.

we show one such method of reading any data below:

**Impact:** Anyone can read the private password, severely breaking the functionality of the protocol

### Proof of Concept: (Proof of Code)

The below test case shows that anyone can read the password from the blockchain

- ## 1. Create a locally running chain

```
1 make anvil
```

- ## 2. Deploy the contract to the chain

```
1 make deploy
```

- ### 3. Run the storage tool

We use 1 because that's the storage slot of `s_password` in the contract.

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

[illegible]

You can pass the hex to string with

[illegible]

and get an output of

```
1 myPassword
```

**Recommended Mitigation:** To maintain confidentiality, passwords should not be stored in plaintext on-chain. Consider encrypting the password off-chain and storing only the ciphertext on-chain. This, however, introduces new complexities, such as key management and off-chain decryption.

Also, remove the getter function entirely if its purpose contradicts security principles or leads to accidental leakage.

---

## Likelihood & impact:

- Impact: High
- Likelihood: High
- Severity: High

### [H-2] Password can be set by anyone which can lead to unchecked access control

**Description:** `setPassword` function allows anyone to set password

**Impact:** Anyone can set the password which contradicts the intended roles

**Proof of Concept:** Add the following to the passwordstore `PasswordStore.t.sol`

Code

```
1 function testAnyOneCanSetPassword(address randomAddress) public {
2     vm.assume(randomAddress != owner);
3     vm.prank(randomAddress);
4     string memory expectedPassword = "myNewPassword";
5     passwordStore.setPassword(expectedPassword);
6
7     vm.prank(owner);
8     string memory actualPassword = passwordStore.getPassword();
9
10    assertEq(actualPassword, expectedPassword);
11 }
```

**Recommended Mitigation:** Add an access control condition to the `setPassword`

```
1 if (s_owner != msg.sender) {
2     revert PasswordStore__NotOwner();
3 }
```

### [I-1] The PasswordStore::getPassword natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

**Description:** The function should be `getPassword()` but the natspects indicates that it requires an argument i.e `getPassword(string)`

**Impact:** Misleading natspec which can cause confusion

**Recommended Mitigation:** Remove the incorrect natspec line

```
1 - * @param newPassword The new password to set.
```